INBEC · AULA PRÁTICA 4 · PROJETO WEB COMPLETO

Da primeira tela ao primeiro registro no banco.

Manual passo a passo para construir um cadastro com HTML, CSS e JavaScript, conectar a um backend e salvar dados de verdade com SQLite.

Iniciante | Python + Flask ou Node + Express | 23 de outubro de 2026

01 · O sistema que vamos construir

Hoje vamos construir um **cadastro de alunos** com nome, e-mail e curso. O usuário preenche o formulário, o JavaScript envia os dados ao backend e o backend grava no **SQLite**. Ao final, a própria página consulta e exibe os cadastros.

Este é seu primeiro sistema web completo: frontend + API + banco de dados. Não exige React, Expo nem conhecimento das práticas anteriores. Use dados fictícios durante a aula.

> Siga **um backend por vez**: Python com Flask é o percurso principal. Node.js com Express é uma alternativa completa ao final. Os dois usam os mesmos três arquivos do frontend e o mesmo formato de tabela.

Escopo: criar e consultar registros (as operações C e R de CRUD). Editar e excluir ficam como desafio orientado. Cadastro de pessoas não é cadastro de contas: não implementaremos senha ou login.

02 · Roteiro de trabalho

Tempo sugerido	Etapa	Evidência
0–20 min	Compreender as três camadas e conferir ferramentas	Explicar o caminho dos dados
20–45 min	Criar pastas, ambiente e backend	API retorna uma lista vazia

45–90 min	Construir HTML, CSS e JavaScript	Formulário envia um cadastro
90–120 min	Ler SQL, validar e testar erros	E-mail repetido retorna 409
120–150 min	Verificar persistência e personalizar	Registro continua após reiniciar
150–180 min	Revisar, explicar e entregar	Código e evidências

Em dupla, alternem entre quem digita e quem confere a cada arquivo. Se dividir em dois encontros, encerre o primeiro com o formulário e o backend funcionando; no segundo, retome os testes de banco e a personalização.

03 · Entender frontend, backend e banco

> **HTML + CSS + JS no navegador**
>
> **↓ POST /api/alunos com JSON**
>
> **Python/Flask ou Node/Express**
>
> **↓ INSERT com parâmetros**
>
> **Arquivo cadastros.db (SQLite)**
>
> **↑ SELECT → JSON → lista na tela**

- **HTML:** estrutura, campos, rótulos e botões.
- **CSS:** aparência, espaçamento e adaptação ao celular.
- **JavaScript:** intercepta o envio e usa fetch para conversar com a API.
- **Backend:** recebe HTTP, valida os dados e executa SQL.
- **SQLite:** motor de banco que grava em um arquivo local. SQL é a linguagem das consultas.

O navegador não abre o arquivo .db. Somente o backend acessa o banco. JSON é o formato de troca de dados, não o banco de dados. API é a interface HTTP que combina endereço, método e resposta.

Método e rota	Entrada	Saída
GET /	Nenhuma	Página HTML

GET /api/alunos	Nenhuma	200 e uma lista JSON
POST /api/alunos	JSON: nome, email, curso	201 e o novo aluno; 400 para dados inválidos; 409 para e-mail repetido

04 · Preparar as ferramentas

Instale VS Code e **Python 3.11 ou superior**. No Windows, confira Python no terminal do VS Code (Terminal → Novo Terminal):

Terminal — execute uma linha por vez

```
py --version
py -m pip --version
```

Se o comando py não existir, tente `python --version` e substitua py por python nas instruções. Se nenhum funcionar, instale Python pelo site oficial e reabra o VS Code. SQLite já acompanha a biblioteca padrão do Python: não instale um pacote chamado sqlite pelo pip.

Para o percurso alternativo, use **Node.js 24.x**, que dispõe de node:sqlite, e confira `node --version` e `npm --version` . Não precisa instalar Node para fazer o percurso Python.

Ferramentas opcionais: DB Browser for SQLite para visualizar tabelas; DevTools do navegador (F12) para inspecionar rede e console.

05 · Criar a pasta do projeto

Crie uma pasta nova para o sistema dos alunos, separada dos manuais. Abra essa pasta no VS Code. No PowerShell:

Terminal — execute uma linha por vez

```
mkdir cadastro-alunos
cd cadastro-alunos
mkdir public
```

Estrutura do percurso Python

```
cadastro-alunos/
  app.py
  requirements.txt
  .gitignore
  public/
    index.html
    style.css
    script.js
  cadastros.db       # criado automaticamente ao iniciar o backend
  .venv/            # criado pelo comando de ambiente virtual
```

Use exatamente esses nomes, inclusive `public`. O index.html deste manual é material de leitura. O `public/index.html` que você criará é a tela do sistema.

A pasta exemplo/public acompanha o manual para consulta dos arquivos, mas abrir seu HTML diretamente não liga um servidor. Execute o backend antes de usar o sistema.

06 · Criar o ambiente Python

Na raiz de cadastro-alunos, crie requirements.txt e .gitignore com estes conteúdos:

Copie o conteúdo completo, sem as marcações do manual. Baixar requirements.txt

requirements.txt

```
Flask>=3.1,<4
```

Copie o conteúdo completo, sem as marcações do manual. Baixar .gitignore

.gitignore

```
.venv/
node_modules/
__pycache__/
*.db
*.db-journal
*.db-wal
*.db-shm
```

Terminal — execute uma linha por vez

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Resultado esperado: instalação do Flask sem erro. Usaremos o executável do ambiente diretamente, sem ativação; isso evita problemas de política do PowerShell. Não copie a pasta .venv entre computadores.

▶ Linux e macOS

07 · Desenhar o banco e compreender SQL

A tabela alunos tem uma linha por cadastro. `id` identifica o aluno; `email` não pode se repetir. `COLLATE NOCASE` ajuda a tratar diferenças de maiúsculas no e-mail ASCII. A aplicação também normaliza o e-mail para minúsculas.

SQL explicativo — o backend executará essas instruções

```
CREATE TABLE IF NOT EXISTS alunos (
  id INTEGER PRIMARY KEY,
  nome TEXT NOT NULL CHECK(length(trim(nome)) BETWEEN 2 AND 80),
  email TEXT NOT NULL UNIQUE COLLATE NOCASE,
  curso TEXT NOT NULL CHECK(length(trim(curso)) BETWEEN 2 AND 80)
);

INSERT INTO alunos (nome, email, curso) VALUES (?, ?, ?);
SELECT id, nome, email, curso FROM alunos ORDER BY id DESC;
```

Não execute esse bloco no PowerShell. CREATE prepara a tabela sem apagá-la se já existir; INSERT grava; SELECT consulta. O id é atribuído pelo SQLite quando omitido. Não presuma numeração sem intervalos.

Os pontos de interrogação recebem parâmetros separados do texto SQL. Assim um nome como `Ana d'Ávila` continua sendo um valor. Não concatene os campos digitados em comandos SQL. O commit confirma a gravação; no Python o bloco `with banco` confirma ao terminar com sucesso e reverte em caso de erro.

08 · Escrever o backend Python completo

Copie o conteúdo completo, sem as marcações do manual. Baixar app.py

app.py

```
from pathlib import Path
import re
import sqlite3
from contextlib import closing
from flask import Flask, jsonify, request

BASE = Path(__file__).resolve().parent
DATABASE = BASE / 'cadastros.db'
app = Flask(__name__, static_folder='public', static_url_path='')
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024


def conectar():
    banco = sqlite3.connect(DATABASE)
    banco.row_factory = sqlite3.Row
    return banco


def preparar_banco():
    with closing(conectar()) as banco, banco:
        banco.execute('''CREATE TABLE IF NOT EXISTS alunos (
            id INTEGER PRIMARY KEY,
            nome TEXT NOT NULL CHECK(length(trim(nome)) BETWEEN 2 AND 80),
            email TEXT NOT NULL UNIQUE COLLATE NOCASE,
            curso TEXT NOT NULL CHECK(length(trim(curso)) BETWEEN 2 AND 80)
        )''')


@app.get('/')
def inicio():
    return app.send_static_file('index.html')


@app.get('/api/alunos')
def listar():
    with closing(conectar()) as banco:
        alunos = banco.execute('SELECT id, nome, email, curso FROM alunos ORDER BY id
DESC').fetchall()
    return jsonify([dict(aluno) for aluno in alunos])


@app.post('/api/alunos')
def cadastrar():
    dados = request.get_json(silent=True)
    if not isinstance(dados, dict):
        return jsonify(erro='Envie um objeto JSON válido.'), 400
    if any(not isinstance(dados.get(campo), str) for campo in ('nome', 'email', 'curso')):
        return jsonify(erro='Nome, e-mail e curso devem ser textos.'), 400
    nome, email, curso = (dados[campo].strip() for campo in ('nome', 'email', 'curso'))
```

```
    email = email.lower()
    if not (2 <= len(nome) <= 80 and 2 <= len(curso) <= 80):
        return jsonify(erro='Nome e curso precisam ter de 2 a 80 caracteres.'), 400
    if len(email) > 120 or not re.fullmatch(r'[^\s@]+@[^\s@]+\.[^\s@]+', email):
        return jsonify(erro='Informe um e-mail válido com até 120 caracteres.'), 400
    try:
        with closing(conectar()) as banco, banco:
            cursor = banco.execute('INSERT INTO alunos (nome, email, curso) VALUES (?, ?, ?)',
                                   (nome, email, curso))
            aluno_id = cursor.lastrowid
    except sqlite3.IntegrityError:
        return jsonify(erro='Este e-mail já está cadastrado.'), 409
    return jsonify(id=aluno_id, nome=nome, email=email, curso=curso), 201


@app.errorhandler(413)
def muito_grande(erro):
    return jsonify(erro='O corpo da requisição excedeu o limite.'), 413


if __name__ == '__main__':
    preparar_banco()
    app.run(host='127.0.0.1', port=3000, debug=False)
```

Leia o código antes de seguir

- Path fixa o caminho do banco ao lado de app.py, mesmo se o comando partir de outra pasta.
- conectar cria uma conexão; closing garante seu fechamento.
- preparar_banco cria a tabela na inicialização.
- app.get e app.post associam funções às rotas HTTP.
- get_json recebe o objeto enviado pelo navegador; a validação do servidor funciona mesmo sem o formulário.
- jsonify transforma os dados em resposta JSON.
- O bloco try trata a restrição de e-mail único; a transação evita gravação parcial.
- public é a única pasta de arquivos estáticos. O arquivo .db fica fora dela.

09 · Iniciar e testar a API antes da tela

Terminal — execute uma linha por vez

```
.\.venv\Scripts\python.exe app.py
```

Deixe o terminal aberto. O endereço será http://127.0.0.1:3000/api/alunos. Abra no navegador: deve aparecer `[]` se o banco é novo. Confira também que cadastros.db apareceu ao lado de app.py.

Nesse momento, acessar / pode dar 404 se o HTML ainda não existe. Isso será resolvido ao criar o próximo arquivo. Para parar o servidor, pressione Ctrl+C no terminal. Não use dois backends na porta 3000 ao mesmo tempo.

10 · Criar o formulário HTML

Copie o conteúdo completo, sem as marcações do manual. Baixar public/index.html

public/index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Cadastro de alunos</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main>
    <header><p class="tag">MEU PRIMEIRO SISTEMA</p><h1>Cadastro de alunos</h1><p>Da tela ao banco de
dados.</p></header>
    <section aria-labelledby="titulo-form"><h2 id="titulo-form">Novo aluno</h2>
      <form id="cadastro">
        <label for="nome">Nome</label>
        <input id="nome" name="nome" required minlength="2" maxlength="80" autocomplete="name">
        <label for="email">E-mail</label>
        <input id="email" name="email" type="email" required maxlength="120" autocomplete="email">
        <label for="curso">Curso</label>
        <input id="curso" name="curso" required minlength="2" maxlength="80">
        <button id="salvar" type="submit">Cadastrar aluno</button>
      </form>
      <p id="mensagem" role="status" aria-live="polite"></p>
    </section>
    <section aria-labelledby="titulo-lista"><h2 id="titulo-lista">Alunos cadastrados</h2>
      <button type="button" id="atualizar">Atualizar lista</button>
      <ul id="lista"></ul>
    </section>
  </main>
```

```
</body>
</html>
```

Os atributos name determinam as chaves do JSON. label e for relacionam o nome do campo ao input. required, minlength e type=email ajudam o usuário; não substituem as verificações do backend. defer aguarda o HTML antes de executar o JavaScript. role=status permite anunciar mensagens.

11 · Estilizar com CSS

Copie o conteúdo completo, sem as marcações do manual. Baixar public/style.css

public/style.css

```
* { box-sizing: border-box; }
body { margin: 0; background: #edf2f7; color: #20324a; font: 16px/1.6 system-ui, sans-serif; }
main { max-width: 760px; margin: auto; padding: 32px 18px; }
header { margin-bottom: 24px; }
h1 { font-size: clamp(28px, 6vw, 42px); line-height: 1.2; }
h2 { margin-top: 0; }
.tag { color: #176254; font-size: 12px; font-weight: bold; letter-spacing: 2px; }
section { padding: 24px; margin-bottom: 20px; background: white; border-radius: 14px; border: 1px
solid #d5dfeb; }
form { display: grid; gap: 8px; }
label { font-weight: 600; }
input { width: 100%; padding: 12px; margin-bottom: 10px; border: 1px solid #8293a7; border-radius:
6px; font: inherit; }
button { padding: 12px 18px; background: #176254; color: white; border: 0; border-radius: 6px; font:
inherit; cursor: pointer; }
button:disabled { opacity: .6; cursor: wait; }
:focus-visible { outline: 3px solid #d28a21; outline-offset: 3px; }
ul { padding: 0; list-style: none; }
li { padding: 14px 0; border-bottom: 1px solid #d5dfeb; overflow-wrap: anywhere; }
#mensagem { min-height: 26px; font-weight: 600; }
#mensagem[data-erro="true"] { color: #aa2525; }
```

O layout usa uma coluna com largura máxima e margens automáticas. box-sizing inclui bordas no cálculo das dimensões. O foco visível ajuda na navegação com teclado. Reduza a janela e verifique se os campos continuam cabendo.

12 · Conectar o frontend à API

Copie o conteúdo completo, sem as marcações do manual. Baixar public/script.js

public/script.js

```
const form = document.querySelector('#cadastro');
const lista = document.querySelector('#lista');
const mensagem = document.querySelector('#mensagem');
const salvar = document.querySelector('#salvar');

function avisar(texto, erro = false) {
  mensagem.textContent = texto;
  mensagem.dataset.erro = String(erro);
}

async function listar() {
  const resposta = await fetch('/api/alunos');
  if (!resposta.ok) throw new Error('Não foi possível carregar a lista.');
  const alunos = await resposta.json();
  lista.replaceChildren();
  if (alunos.length === 0) {
    const item = document.createElement('li');
    item.textContent = 'Nenhum aluno cadastrado ainda.';
    lista.append(item);
  }
  for (const aluno of alunos) {
    const item = document.createElement('li');
    item.textContent = `${aluno.nome} — ${aluno.email} — ${aluno.curso}`;
    lista.append(item);
  }
}

form.addEventListener('submit', async (evento) => {
  evento.preventDefault();
  if (salvar.disabled) return;
  salvar.disabled = true;
  avisar('Salvando…');
  try {
    const dados = Object.fromEntries(new FormData(form));
    const resposta = await fetch('/api/alunos', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(dados),
    });
    const resultado = await resposta.json();
    if (!resposta.ok) throw new Error(resultado.erro || 'Não foi possível cadastrar.');
    form.reset();
    avisar('Aluno cadastrado com sucesso!');
```

```
    try { await listar(); }
    catch { avisar('Cadastro salvo, mas a lista não atualizou. Clique em Atualizar lista.', true); }
  } catch (erro) {
    avisar(erro.message || 'Confira se o servidor está ligado.', true);
  } finally {
    salvar.disabled = false;
  }
});

document.querySelector('#atualizar').addEventListener('click', () => {
  listar().then(() => avisar('Lista atualizada.')).catch(erro => avisar(erro.message, true));
});
listar().catch(() => avisar('Não foi possível carregar. Confira se o servidor está ligado.', true));
```

Como o envio funciona

1. submit dispara quando o formulário é enviado; preventDefault impede a navegação padrão.
2. FormData coleta os campos e Object.fromEntries monta um objeto.
3. JSON.stringify transforma o objeto em texto JSON.
4. fetch envia POST e Content-Type informa o formato ao servidor.
5. await aguarda cada resposta; resposta.ok verifica códigos de sucesso. fetch não lança erro automaticamente em um HTTP 400 ou 409.
6. Ao receber 201, o formulário limpa e a lista é consultada novamente.
7. textContent exibe os nomes como texto, sem interpretar HTML digitado pelo usuário.
8. finally reabilita o botão mesmo se ocorrer uma falha.

O endereço relativo /api/alunos usa o mesmo servidor que entregou o HTML. Assim não precisamos de CORS nem de duas portas. Não abra esta tela por file:// ou Live Server.

13 · Executar o sistema completo

1. Salve todos os arquivos. Se o servidor já estiver ligado, mantenha-o; se alterou app.py, pare e inicie novamente.
2. Abra http://127.0.0.1:3000.
3. Cadastre Ana Souza, ana@example.com, Sistemas de Informação.
4. Confira a mensagem de sucesso e o aluno na lista.
5. Recarregue a página: o aluno deve continuar.
6. Cadastre outro aluno usando outro e-mail.

> GitHub Pages pode hospedar este manual e os arquivos de exemplo. Ele não executa Python, Node nem o banco SQLite do sistema. O sistema desta aula roda localmente na máquina de cada aluno.

14 · Ver a comunicação e inspecionar o banco

Pressione F12 → Rede/Network. Cadastre um aluno com e-mail novo e selecione a requisição POST /api/alunos:

- Request/Payload: veja nome, email e curso no JSON.
- Status: 201 Created.
- Response: objeto do aluno com id.
- Em seguida, GET /api/alunos retorna 200 e a lista.

Em um segundo terminal PowerShell, na raiz do projeto:

Terminal — execute uma linha por vez

```
Invoke-RestMethod http://127.0.0.1:3000/api/alunos
$aluno = @{ nome = "Bruno Lima"; email = "bruno@example.com"; curso = "Computação" } | ConvertTo-Json
Invoke-RestMethod -Uri http://127.0.0.1:3000/api/alunos -Method Post -ContentType "application/json;
charset=utf-8" -Body ([System.Text.Encoding]::UTF8.GetBytes($aluno))
```

Repetir o segundo envio deve retornar erro 409: isso é um teste de duplicidade, não uma instalação quebrada.

Conferir o arquivo SQLite com Python

Opcional: crie consultar.py na raiz do sistema

```
import sqlite3
from pathlib import Path

caminho = Path(__file__).resolve().parent / "cadastros.db"
conexao = sqlite3.connect(f"{caminho.as_uri()}?mode=ro", uri=True)
try:
    for linha in conexao.execute("SELECT id, nome, email, curso FROM alunos"):
        print(linha)
finally:
    conexao.close()
```

Terminal — execute uma linha por vez

```
.\.venv\Scripts\python.exe consultar.py
```

O modo somente leitura falha se o banco não existe, evitando criar um arquivo vazio no caminho errado. Como alternativa, abra cadastros.db no DB Browser e use Browse Data. Feche transações de edição antes de continuar os testes.

15 · Checklist de funcionamento e persistência

- [ ] API vazia retorna [] antes do primeiro cadastro.
- [ ] Cadastro válido aparece na tela e recebe id.
- [ ] Campo vazio é bloqueado no formulário.
- [ ] Envio direto de JSON com nome vazio recebe 400.
- [ ] E-mail repetido recebe 409 sem criar outra linha.
- [ ] Nome com apóstrofo é salvo corretamente.
- [ ] Texto como Ana aparece como texto na lista.
- [ ] Depois de Ctrl+C e reiniciar o backend, os dados permanecem.
- [ ] O mesmo cadastro pode ser confirmado no arquivo SQLite.
- [ ] Tab percorre os campos; Enter envia o formulário.

Para testar validação sem depender do HTML, envie pelo PowerShell um objeto com nome vazio ou troque nome por um número. Aguarde HTTP 400. Registre resultado esperado e observado; não basta uma captura da página aberta.

16 · Alternativa completa: backend Node.js

Use esta seção se optar por Node no lugar de Python. Preserve os mesmos arquivos de public e .gitignore. Não é necessário executar app.py, instalar Flask ou criar .venv. Instale Node **24.x**, confira sua versão e crie server.cjs na raiz. A extensão cjs permite usar require independentemente do tipo configurado no package.json.

Terminal — execute uma linha por vez

```
node --version
npm.cmd --version
npm.cmd init -y
npm.cmd install express@5
```

npm.cmd é útil no PowerShell quando a execução de npm.ps1 está bloqueada. No Linux/macOS, use npm. O comando init cria package.json; install cria node_modules e package-lock.json. O SQLite vem de node:sqlite; não instale sqlite3 para este exemplo.

Copie o conteúdo completo, sem as marcações do manual. Baixar server.cjs

server.cjs

```
const express = require('express');
const path = require('node:path');
const { DatabaseSync } = require('node:sqlite');
const app = express();
const db = new DatabaseSync(path.join(__dirname, 'cadastros.db'));
db.exec(`CREATE TABLE IF NOT EXISTS alunos (
  id INTEGER PRIMARY KEY,
  nome TEXT NOT NULL CHECK(length(trim(nome)) BETWEEN 2 AND 80),
  email TEXT NOT NULL UNIQUE COLLATE NOCASE,
  curso TEXT NOT NULL CHECK(length(trim(curso)) BETWEEN 2 AND 80)
)`);
app.use(express.json({ limit: '16kb' }));
app.use(express.static(path.join(__dirname, 'public')));
app.get('/api/alunos', (req, res) => {
  res.json(db.prepare('SELECT id, nome, email, curso FROM alunos ORDER BY id DESC').all());
});
app.post('/api/alunos', (req, res) => {
  const dados = req.body;
  if (!dados || ['nome', 'email', 'curso'].some(campo => typeof dados[campo] !== 'string')) {
    return res.status(400).json({ erro: 'Nome, e-mail e curso devem ser textos.' });
  }
  const nome = dados.nome.trim();
  const email = dados.email.trim().toLowerCase();
  const curso = dados.curso.trim();
  if (nome.length < 2 || nome.length > 80 || curso.length < 2 || curso.length > 80) {
    return res.status(400).json({ erro: 'Nome e curso precisam ter de 2 a 80 caracteres.' });
  }
  if (email.length > 120 || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    return res.status(400).json({ erro: 'Informe um e-mail válido com até 120 caracteres.' });
  }
  try {
    const resultado = db.prepare('INSERT INTO alunos (nome, email, curso) VALUES (?, ?, ?)').run(nome,
email, curso);
    res.status(201).json({ id: Number(resultado.lastInsertRowid), nome, email, curso });
  } catch (erro) {
```

```
    if (erro.code?.startsWith('ERR_SQLITE') && erro.message.includes('UNIQUE')) {
      return res.status(409).json({ erro: 'Este e-mail já está cadastrado.' });
    }
    throw erro;
  }
});
app.use((erro, req, res, next) => {
  if (erro.type === 'entity.parse.failed') return res.status(400).json({ erro: 'JSON inválido.' });
  if (erro.type === 'entity.too.large') return res.status(413).json({ erro: 'Corpo muito grande.' });
  console.error(erro);
  res.status(500).json({ erro: 'Erro interno. Confira o terminal do servidor.' });
});
app.listen(3000, '127.0.0.1', () => console.log('Abra http://127.0.0.1:3000'));
```

Terminal — execute uma linha por vez

```
node server.cjs
```

Abra o mesmo endereço http://127.0.0.1:3000. Repita todos os testes da seção anterior. Pare Python antes de iniciar Node. Os dois usam o mesmo cadastros.db quando os arquivos estão na mesma pasta; isso permite comparar implementações sem apagar os registros.

DatabaseSync executa operações síncronas: é uma escolha simples para uma aula local com poucos dados. As consultas usam prepare e parâmetros; Express recebe JSON e serve public. O middleware final converte erros em respostas JSON.

Ao retomar em outro computador, leve package.json e package-lock.json e execute npm.cmd ci. Não envie node_modules nem um banco contendo dados pessoais ao repositório.

17 · Diagnosticar os erros mais comuns

Sintoma	O que conferir
Connection refused / site não abre	Backend precisa ficar executando no terminal; abra a porta 3000 indicada.
Failed to fetch / JSON inesperado	Use http://127.0.0.1:3000, não file://, Live Server ou o endereço do manual.
No module named flask	Instale e execute usando o mesmo Python da .venv.

Arquivo não encontrado	Terminal deve estar na pasta do projeto. Confira app.py e public/index.html; evite index.html.txt.
404 no CSS/JS	Confira style.css e script.js dentro de public e veja Rede no navegador.
Address already in use / EADDRINUSE	Pare o outro servidor com Ctrl+C. Python e Node são alternativas na mesma porta.
409 ao cadastrar	O e-mail já existe. Use outro ou apenas consulte os dados.
400 ao cadastrar	Confira tipos, espaços, limites e formato de e-mail; leia a mensagem JSON.
database is locked	Feche transações pendentes no visualizador; evite editar o banco durante o teste.
Dados desapareceram	Confira se iniciou outra cópia do projeto e qual arquivo .db está ao lado do backend.
node:sqlite não encontrado	Confira node --version. Esta alternativa foi escrita para Node 24.x.
Alterei o código, mas nada mudou	Salve, reinicie o backend se necessário e recarregue com Ctrl+F5.

Procedimento: leia primeiro o erro no terminal, depois Console e Rede do navegador. Identifique se a falha ocorre na tela, no HTTP ou no SQL antes de alterar o código.

18 · Atividade, desafios e entrega

Atividade principal

Personalize o sistema para cadastro de participantes de uma oficina. Mude textos, cores e exemplos sem quebrar o contrato nome/email/curso. Cadastre três registros fictícios e demonstre a persistência.

Desafios progressivos

1. Adicionar contador de alunos usando o tamanho da lista.
2. Filtrar os alunos na tela por nome usando os dados já consultados.

3. Adicionar telefone: atualizar HTML, validação, SQL e resposta. Uma tabela existente não ganha a coluna só por alterar CREATE TABLE IF NOT EXISTS: planeje uma migração com ALTER TABLE e backup.
4. Implementar DELETE /api/alunos/:id com confirmação e DELETE FROM alunos WHERE id = ?. Validar id e retornar 404 se não existir.
5. Implementar edição com PUT, validar os mesmos campos e atualizar com WHERE id = ?. Testar duplicidade e registro inexistente.

Entrega

Entregue public com os três arquivos, o backend escolhido e os arquivos de dependências correspondentes. Inclua README com como instalar, iniciar, abrir e testar. Não entregue .venv, node_modules ou dados pessoais reais. Anexe evidências de cadastro, erro de duplicidade e persistência após reinício.

Critério	Pontos
Formulário acessível e CSS adaptável	2
JavaScript integra POST e GET e trata erros	2
Backend valida e responde com status adequados	2
SQLite grava com parâmetros e persiste após reinício	2
Explicação do fluxo e evidências de testes	2

Explique com suas palavras: por que o frontend não acessa diretamente o banco? O que muda entre 200, 201, 400 e 409? Por que validar no servidor? Onde está o commit? Qual arquivo precisa continuar existindo para preservar os registros?

19 · Encerrar e retomar com segurança

Encerre o backend com Ctrl+C. Para retomar, abra a mesma pasta, execute o comando de inicialização e visite a porta 3000. Não recrie o banco: a tabela já existe e os registros serão consultados.

Este exemplo é local e didático. Antes de publicar um sistema real, será necessário projetar autenticação e autorização, proteção dos dados, HTTPS, backups, implantação do backend e armazenamento persistente. O servidor Flask desta aula é de desenvolvimento. Não mova cadastros.db para public.

20 · Referências oficiais

- MDN: formulários HTML
- MDN: uso de fetch
- Flask: conexão e banco de dados
- Python: sqlite3, parâmetros e transações
- Node 24: node:sqlite e DatabaseSync
- Express 5: API
- SQLite: linguagem SQL

Material preparado para a aula de 23 de outubro de 2026. As instruções usam Python/Flask no percurso principal e Node 24/Express 5 no alternativo.